SQL
SELECT

Ing. Ladislav Körösi, PhD.
ÚRK, FEI STU

ladislav.korosi@stuba.sk

TABLE Filmy

ID	NAZOV	ROK	ZANER	IMDB
1	ĽUDIA AKO MY	1999	dráma	7.1
2	NOMÁD	2005	historický	7.1
3	IRON MAN 2	2010	akčný	7.0
4	IRON MAN	2008	akčný	7.9
5	SYNOVEC	1998	romantický	8.0
6	MILIONÁR Z CHATRČE	2008	romantický	8.0
7	WINNETOU A MIEŠANKA APANAČI	1966	western	5.5
8	SPARTAKUS	2004	historický	6.8
9	IRON MAN 3	2013	akčný	7.0

SELECT

The SELECT statement is used to select data from a database.

The result is stored in a result table, called the result-set.

Syntax:

```
SELECT field1, field2,...fieldN table_name1, table_name2...
[WHERE Clause]
[OFFSET M ][LIMIT N]
```

SELECT

- You can use one or more tables separated by comma to include various conditions using a WHERE clause, but WHERE clause is an optional part of SELECT command.
- You can specify star (*) in place of fields. In this case, SELECT will return all the fields.
- You can specify any condition using WHERE clause.
- You can specify an offset using OFFSET from where SELECT will start returning records. By default offset is zero.
- You can limit the number of returns using LIMIT attribute.

SELECT

Example:

```
SELECT * FROM `filmy`
```

Selects all the columns from the „Filmy" table

ID	NAZOV	ROK	ZANER	IMDB
1	ĽUDIA AKO MY	1999	dráma	7.1
2	NOMÁD	2005	historický	7.1
3	IRON MAN 2	2010	akčný	7.0
4	IRON MAN	2008	akčný	7.9
5	SYNOVEC	1998	romantický	8.0
6	MILIONÁR Z CHATRČE	2008	romantický	8.0
7	WINNETOU A MIEŠANKA APANAČI	1966	western	5.5
8	SPARTAKUS	2004	historický	6.8
9	IRON MAN 3	2013	akčný	7.0

SELECT

Example:

```
SELECT ID, NAZOV, ROK, ZANER, IMDB FROM `filmy`
```

Selects ID, ... , IMDB columns from the „Filmy" table

ID	NAZOV	ROK	ZANER	IMDB
1	ĽUDIA AKO MY	1999	dráma	7.1
2	NOMÁD	2005	historický	7.1
3	IRON MAN 2	2010	akčný	7.0
4	IRON MAN	2008	akčný	7.9
5	SYNOVEC	1998	romantický	8.0
6	MILIONÁR Z CHATRČE	2008	romantický	8.0
7	WINNETOU A MIEŠANKA APANAČI	1966	western	5.5
8	SPARTAKUS	2004	historický	6.8
9	IRON MAN 3	2013	akčný	7.0

SELECT

Example:

```
SELECT ID, ROK, NAZOV, ZANER, IMDB FROM `filmy`
```

Selects ID, ... , IMDB columns from the „Filmy" table

ID	ROK	NAZOV	ZANER	IMDB
1	1999	ĽUDIA AKO MY	dráma	7.1
2	2005	NOMÁD	historický	7.1
3	2010	IRON MAN 2	akčný	7.0
4	2008	IRON MAN	akčný	7.9
5	1998	SYNOVEC	romantický	8.0
6	2008	MILIONÁR Z CHATRČE	romantický	8.0
7	1966	WINNETOU A MIEŠANKA APANAČI	western	5.5
8	2004	SPARTAKUS	historický	6.8
9	2013	IRON MAN 3	akčný	7.0

SELECT

Example:

```
SELECT ZANER FROM `filmy`
```

Selects ZANER columns from the „Filmy" table

ZANER
dráma
historický
akčný
akčný
romantický
romantický
western
historický
akčný

SELECT DISTINCT

Example:

```
SELECT DISTINCT ZANER FROM `filmy`
```

The SELECT DISTINCT statement is used to return only distinct (different) values.

ZANER
dráma
historický
akčný
romantický
western

SELECT DISTINCT

Example:

```
SELECT ZANER, IMDB FROM `filmy`
```

ZANER	IMDB
dráma	7.1
historický	7.1
akčný	7.0
akčný	7.9
romantický	8.0
romantický	8.0
western	5.5
historický	6.8
akčný	7.0

```
SELECT DISTINCT ZANER, IMDB FROM `filmy`
```

ZANER	IMDB
dráma	7.1
historický	7.1
akčný	7.0
akčný	7.9
romantický	8.0
western	5.5
historický	6.8

SELECT WHERE

The WHERE clause is used to extract only those records that fulfill a specified criterion.

```
SELECT column_name,column_name
FROM table_name
WHERE column_name operator value;
```

SELECT WHERE

Operator	Description
=	Equal
<>	Not equal. **Note:** In some versions of SQL this operator may be written as !=
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
BETWEEN	Between an inclusive range
LIKE	Search for a pattern
IN	To specify multiple possible values for a column

SELECT WHERE (=, >)

Example:

```
SELECT * FROM `filmy` WHERE rok = 2008
```

ID	NAZOV	ROK	ZANER	IMDB
4	IRON MAN	2008	akčný	7.9
6	MILIONÁR Z CHATRČE	2008	romantický	8.0

Example:

```
SELECT * FROM `filmy` WHERE rok > 2008
```

ID	NAZOV	ROK	ZANER	IMDB
3	IRON MAN 2	2010	akčný	7.0
9	IRON MAN 3	2013	akčný	7.0

SELECT WHERE (=)

Example:

```
SELECT * FROM `filmy` WHERE zaner = 'akčný'
```

ID	NAZOV	ROK	ZANER	IMDB
3	IRON MAN 2	2010	akčný	7.0
4	IRON MAN	2008	akčný	7.9
9	IRON MAN 3	2013	akčný	7.0

SELECT WHERE (<>, !=)

Example:

```
SELECT * FROM `filmy` WHERE zaner <> 'akčný'
```

ID	NAZOV	ROK	ZANER	IMDB
1	ĽUDIA AKO MY	1999	dráma	7.1
2	NOMÁD	2005	historický	7.1
5	SYNOVEC	1998	romantický	8.0
6	MILIONÁR Z CHATRČE	2008	romantický	8.0
7	WINNETOU A MIEŠANKA APANAČI	1966	western	5.5
8	SPARTAKUS	2004	historický	6.8

SELECT WHERE (LIKE)

The LIKE operator is used to search for a specified pattern in a column

Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE column_name LIKE pattern;
```

The "%" sign is used to define wildcards (missing letters) both before and after the pattern.

SELECT WHERE (LIKE)

The following SQL statement selects all customers with a City starting with the letter "s":

```
SELECT * FROM Customers
WHERE City LIKE 's%';
```

The following SQL statement selects all customers with a City ending with the letter "s":

```
SELECT * FROM Customers
WHERE City LIKE '%s';
```

SELECT WHERE (LIKE)

The following SQL statement selects all customers with a Country containing the pattern "land":

```
SELECT * FROM Customers
WHERE City LIKE '%land%';
```

Using the NOT keyword allows you to select records that do NOT match the pattern. The following SQL statement selects all customers with Country NOT containing the pattern "land":

```
SELECT * FROM Customers
WHERE City NOT LIKE ''%land%';
```

SELECT WHERE (LIKE)

In SQL, wildcard characters are used with the SQL LIKE operator. SQL wildcards are used to search for data within a table. With SQL, the wildcards are:

Wildcard	**Description**
%	A substitute for zero or more characters
_	A substitute for a single character
[*charlist*]	Sets and ranges of characters to match
[^*charlist*] or [!*charlist*]	Matches only a character NOT specified within the brackets

SELECT WHERE (LIKE)

The following SQL statement selects all customers with a City starting with "L", followed by any character, followed by "n", followed by any character, followed by "on":

```
SELECT * FROM Customers
WHERE City LIKE 'L_n_on';
```

The following SQL statement selects all customers with a City starting with "b", "s", or "p":

```
SELECT * FROM Customers
WHERE City LIKE '[bsp]%';
```

SELECT WHERE (LIKE)

The following SQL statement selects all customers with a City starting with "a", "b", or "c":

```
SELECT * FROM Customers
WHERE City LIKE '[a-c]%';
```

SELECT WHERE (LIKE)

The following SQL statement selects all customers with a City NOT starting with "b", "s", or "p":

```
SELECT * FROM Customers
WHERE City LIKE '[!bsp]%';
```

or

```
SELECT * FROM Customers
WHERE City NOT LIKE '[bsp]%';
```

SELECT WHERE (LIKE)

Example:

```
SELECT * FROM `filmy` WHERE zaner like '%est%'
```

ID	NAZOV	ROK	ZANER	IMDB
7	WINNETOU A MIEŠANKA APANAČI	1966	western	5.5

TABLE Filmy

ID	NAZOV	ROK	ZANER	IMDB
1	ĽUDIA AKO MY	1999	dráma	7.1
2	NOMÁD	2005	historický	7.1
3	IRON MAN 2	2010	akčný	7.0
4	IRON MAN	2008	akčný	7.9
5	SYNOVEC	1998	romantický	8.0
6	MILIONÁR Z CHATRČE	2008	romantický	8.0
7	WINNETOU A MIEŠANKA APANAČI	1966	western	5.5
8	SPARTAKUS	2004	historický	6.8
9	IRON MAN 3	2013	akčný	7.0

SELECT WHERE (LIKE)

Example:

```
SELECT * FROM `filmy` WHERE `NAZOV` LIKE 'MI%‘
SELECT * FROM `filmy` WHERE `NAZOV` LIKE ‘mi%‘
```

ID	NAZOV	ROK	ZANER	IMDB
6	MILIONÁR Z CHATRČE	2008	romantický	8.0

SELECT WHERE (BETWEEN)

The BETWEEN operator is used to select values within a range.

Syntax

```
SELECT column_name(s)
FROM table_name
WHERE column_name BETWEEN value1 AND value2;
```

SELECT WHERE (BETWEEN)

Example:

SELECT * FROM `filmy` WHERE ROK BETWEEN 2005 AND 2010

ID	NAZOV	ROK	ZANER	IMDB
1	ĽUDIA AKO MY	1999	dráma	7.1
2	NOMÁD	2005	historický	7.1
5	SYNOVEC	1998	romantický	8.0
6	MILIONÁR Z CHATRČE	2008	romantický	8.0
7	WINNETOU A MIEŠANKA APANAČI	1966	western	5.5
8	SPARTAKUS	2004	historický	6.8

SELECT WHERE (NOT BETWEEN)

Example:

SELECT * FROM `filmy` WHERE ROK NOT BETWEEN 2005 AND 2010

ID	NAZOV	ROK	ZANER	IMDB
1	ĽUDIA AKO MY	1999	dráma	7.1
5	SYNOVEC	1998	romantický	8.0
7	WINNETOU A MIEŠANKA APANAČI	1966	western	5.5
8	SPARTAKUS	2004	historický	6.8
9	IRON MAN 3	2013	akčný	7.0

SELECT WHERE (BETWEEN)

Example:

```
SELECT * FROM filmy WHERE NAZOV BETWEEN 'A' AND 'J'
```

ID	NAZOV	ROK	ZANER	IMDB
3	IRON MAN 2	2010	akčný	7.0
4	IRON MAN	2008	akčný	7.9
9	IRON MAN 3	2013	akčný	7.0

SELECT WHERE (IN)

The IN operator allows you to specify multiple values in a WHERE clause.

Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE column_name IN (value1,value2,...);
```

Example:

```
SELECT * FROM Customers
WHERE City IN ('Paris','London');
```

SELECT WHERE (IN)

The IN operator allows you to specify multiple values in a WHERE clause.

Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE column_name IN (value1,value2,...);
```

Example:

```
SELECT * FROM Customers
WHERE City IN ('Paris','London');
```

SELECT WHERE (IN)

Example:

```
SELECT * FROM `filmy`
WHERE ZANER IN ('akčný','western')
```

ID	NAZOV	ROK	ZANER	IMDB
3	IRON MAN 2	2010	akčný	7.0
4	IRON MAN	2008	akčný	7.9
7	WINNETOU A MIEŠANKA APANAČI	1966	western	5.5
9	IRON MAN 3	2013	akčný	7.0

SELECT WHERE (IN)

Example:

```
SELECT * FROM `filmy`
WHERE ZANER NOT IN ('akčný','western')
```

ID	NAZOV	ROK	ZANER	IMDB
1	ĽUDIA AKO MY	1999	dráma	7.1
2	NOMÁD	2005	historický	7.1
5	SYNOVEC	1998	romantický	8.0
6	MILIONÁR Z CHATRČE	2008	romantický	8.0
8	SPARTAKUS	2004	historický	6.8

SELECT WHERE (IN, LIKE with OR, = with OR)

Example:

```
SELECT * FROM `filmy`
WHERE ZANER IN ('akčný','western')
```

or

```
SELECT * FROM `filmy`
WHERE ZANER LIKE 'akčný' or ZANER LIKE 'western‘
```

or

```
SELECT * FROM `filmy`
WHERE ZANER = 'akčný' or ZANER = 'western'
```

SELECT ORDER BY

The ORDER BY keyword is used to sort the result-set by one or more columns. The ORDER BY keyword sorts the records in ascending order by default. To sort the records in a descending order, you can use the DESC keyword.

Syntax:

```
SELECT column_name, column_name
FROM table_name
ORDER BY column_name ASC|DESC, column_name ASC|DESC;
```

SELECT ORDER BY

Example:

```
SELECT * FROM `filmy` ORDER BY `IMDB` ASC
SELECT * FROM `filmy` ORDER BY filmy.IMDB ASC
```

ID	NAZOV	ROK	ZANER	IMDB ▲ 1
7	WINNETOU A MIEŠANKA APANAČI	1966	western	5.5
8	SPARTAKUS	2004	historický	6.8
3	IRON MAN 2	2010	akčný	7.0
9	IRON MAN 3	2013	akčný	7.0
1	ĽUDIA AKO MY	1999	dráma	7.1
2	NOMÁD	2005	historický	7.1
4	IRON MAN	2008	akčný	7.9
5	SYNOVEC	1998	romantický	8.0
6	MILIONÁR Z CHATRČE	2008	romantický	8.0

SELECT ORDER BY

Example:

```
SELECT * FROM `filmy`
ORDER BY filmy.IMDB ASC, filmy.ROK DESC
```

ID	NAZOV	ROK	ZANER	IMDB ▲ 1
7	WINNETOU A MIEŠANKA APANAČI	1966	western	5.5
8	SPARTAKUS	2004	historický	6.8
9	IRON MAN 3	2013	akčný	7.0
3	IRON MAN 2	2010	akčný	7.0
2	NOMÁD	2005	historický	7.1
1	ĽUDIA AKO MY	1999	dráma	7.1
4	IRON MAN	2008	akčný	7.9
6	MILIONÁR Z CHATRČE	2008	romantický	8.0
5	SYNOVEC	1998	romantický	8.0

SELECT GROUP BY

The GROUP BY statement is used in conjunction with the aggregate functions to group the result-set by one or more columns.

Syntax:

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
WHERE column_name operator value
GROUP BY column_name;
```

SELECT GROUP BY

Example:

```
SELECT zaner FROM `filmy`
```

zaner
dráma
historický
akčný
akčný
romantický
romantický
western
historický
akčný

Example:

```
SELECT zaner FROM `filmy`
GROUP BY ZANER
```

zaner
akčný
dráma
historický
romantický
western

SELECT GROUP BY

Example:

```
SELECT zaner, COUNT(zaner)
FROM `filmy` GROUP BY ZANER
```

zaner	COUNT(zaner)
akčný	3
dráma	1
historický	2
romantický	2
western	1

ALIAS

SQL aliases are used to give a database table, or a column in a table, a temporary name. Basically aliases are created to make column names more readable.

Syntax for columns

```
SELECT column_name AS alias_name
FROM table_name;
```

Syntax for tables

```
SELECT column_name(s)
FROM table_name AS alias_name;
```

ALIAS

Example:

```
SELECT zaner as žáner, COUNT(zaner) as počet
FROM `filmy` GROUP BY ZANER
```

žáner	počet
akčný	3
dráma	1
historický	2
romantický	2
western	1

ALIAS

Example:

In the following SQL statement we combine four columns (Address, City, PostalCode, and Country) and create an alias named "Address":

```
SELECT CustomerName, Address+', '+City+', '+PostalCode+', '+Country AS Address
FROM Customers;
```

```
SELECT CONCAT(CustomerName, Address+', '+City+', '+PostalCode+', '+Country) AS Address
FROM Customers;
```

EXAMPLE

Select all films from table Filmy, where the IMDB is greater than the average (hint avg()).